

GCD Computation Using Euclid's Algorithm

Student Name: Ajaya Kumar Patel

Roll Number: CRS2503

Course: Cryptographic and Security Implementations

July 22, 2026

Contents

1	Introduction	2
2	Euclid's Algorithm	2
2.1	Algorithm	2
3	C Program	3
4	Dry Run	3
5	Complexity Analysis	4
6	Step Count Analysis	4
7	Conclusion	5

1 Introduction

The Greatest Common Divisor of two positive integers is defined to be the greatest positive integer which will divide the two numbers evenly without leaving a remainder. Calculation of the Greatest Common Divisor is a very significant problem in mathematics and computer science. The Greatest Common Divisor is extensively applied in cryptography, especially in RSA cryptography. The best way of calculating the GCD is through use of Euclid's algorithm which involves repeated use of the modulus operator.

2 Euclid's Algorithm

Euclid's Algorithm is based on the following mathematical property:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

The algorithm repeatedly replaces the pair (a, b) with

$$(b, a \bmod b)$$

until the second number becomes zero. The remaining value of a is the Greatest Common Divisor.

2.1 Algorithm

1. Read two positive integers a and b .
2. Repeat the following steps while:

$$b \neq 0$$

- (a) Compute the remainder:

$$r = a \bmod b$$

- (b) Assign:

$$a = b$$

- (c) Assign:

$$b = r$$

3. Return a as the GCD.

3 C Program

Listing 1: GCD Calculation Using Euclidean Algorithm

```
1 #include <stdio.h>
2
3 unsigned int gcd(unsigned int a, unsigned int b)
4 {
5     while (b != 0)
6     {
7         unsigned int r = a % b;
8         a = b;
9         b = r;
10    }
11    return a;
12 }
13
14 int main()
15 {
16     unsigned int a, b;
17
18     printf("Enter two positive integers: ");
19     scanf("%u %u", &a, &b);
20
21     printf("GCD = %u\n", gcd(a, b));
22
23     return 0;
24 }
```

4 Dry Run

Consider the following input:

$$a = 48, \quad b = 18$$

The execution of Euclid's Algorithm is shown below:

Since the remainder becomes zero in the third iteration, the algorithm stops. The value of a at this point is the Greatest Common Divisor.

$$\gcd(48, 18) = 6$$

Iteration	a	b	$a \bmod b$
1	48	18	12
2	18	12	6
3	12	6	0

Table 1: Dry Run of Euclid's Algorithm

5 Complexity Analysis

In cryptography, the input size is measured by the **number of bits** required to represent the input integers rather than their numerical values.

Let the input integers be represented using n bits. Therefore,

$$2^{n-1} \leq a, b < 2^n$$

The number of iterations performed by Euclid's Algorithm is proportional to the bit length of the input. In the worst case, the algorithm requires a linear number of iterations with respect to the input size.

Hence, the time complexity in terms of an n -bit input is:

$$\text{Time Complexity} = O(n)$$

where n represents the number of bits required to represent the input integers.

The algorithm uses only a constant number of variables during execution. Therefore, the memory requirement does not depend on the input size.

Hence, the space complexity is:

$$\text{Space Complexity} = O(1)$$

6 Step Count Analysis

Assume that the algorithm executes k iterations.

Each iteration performs the following basic operations:

1. One comparison operation ($b \neq 0$)
2. One modulo operation ($a \bmod b$)
3. One assignment operation ($a = b$)
4. One assignment operation ($b = r$)
5. One loop control operation

Therefore, approximately 5 basic operations are performed in each iteration.
Since the number of iterations is proportional to the input bit length,

$$k \approx n$$

where n is the number of bits required to represent the input integers.
Therefore, the total number of basic operations can be expressed as:

$$T(n) = 5n + 2$$

where:

- $5n$ represents the operations performed inside the loop.
- 2 represents the initialization and the final return operation.

Hence, the estimated step count is:

$$\boxed{T(n) = 5n + 2}$$

which is linear with respect to the input bit length.

7 Conclusion

Euclid's Algorithm offers an efficient way to find the Greatest Common Divisor of two positive integers. The algorithm decreases the problem size in each step by using the modulo operation. When we look at the input size as an n -bit number, it needs $O(n)$ iterations based on the basic operation model. Because of its efficiency, Euclid's Algorithm is commonly used in cryptography and number theory calculations.